

HiVA: Hierarchical Variable-Horizon Actions for Long-Horizon VLA Control

Jongwoo Park

Abstract

Action chunking improves consistency and amortizes expensive action generation in Vision-Language-Action (VLA) models, but most VLAs still execute a fixed number of micro-actions per model call, creating a brittle trade-off between responsiveness and smooth long-horizon execution. We propose HiVA, a hierarchical variable-horizon action representation for denoising-based VLAs, where each macro action is encoded by a predicted duration and compact B-spline trajectory coefficients that reconstruct smooth, variable-length micro-action sequences. The key idea is to make *how many* micro-actions to execute and *how* those actions are represented part of the same denoising action expert, rather than treating chunk length as a fixed hyperparameter. To make HiVA easy to integrate into existing denoising VLAs, we keep the pretrained multi-modal conditioning path and the main denoising transformer block, and modify only lightweight action-side projection layers and output heads. The method is trained on dynamically re-labeled macro-actions generated by a dynamic-programming oracle that favors segments that are both well approximated by B-splines and aligned with sub-task difficulty. We plan to evaluate the resulting representation on LIBERO-Plus and CALVIN by instantiating it in SmolVLA and VPP, covering different model scales (0.45B, 1.5B) and families (VLM, Diffusion backbones). The intended outcome is a smooth, compact, variable-horizon action representation that transfers across denoising VLA backbones and improves long-horizon manipulation.

1 Related Work

Action chunking and adaptive execution horizons. Action chunking has become a standard design choice in modern robot policies and VLAs because it improves temporal consistency and amortizes expensive action generation over multiple control steps [6, 7, 11, 12]. However, fixed-horizon chunking exposes a persistent trade-off: longer executed chunks improve smoothness and reduce model calls, while shorter chunks improve responsiveness to new observations. Recent inference-time methods address this issue by changing how many actions from a predicted chunk are executed. AAC adapts chunk size using action entropy estimated from multiple sampled chunks, while AutoHorizon adjusts the executed prefix of a denoising-based VLA according to model confidence or attention-derived cues [1, 2]. Real-time chunking

and asynchronous execution methods further reduce pauses and boundary artifacts by generating future chunks while the current chunk is being executed [3, 13]. HiVA is complementary to these inference-time strategies, but differs in that the execution duration is learned as part of the action representation itself. Instead of selecting a prefix from a fixed raw-action chunk after prediction, HiVA predicts a duration-conditioned compact macro action in one denoising pass and decodes it into a variable-length sequence of native micro-actions.

Hierarchical RL and chunked action spaces. The idea of using temporally extended actions is closely related to hierarchical reinforcement learning, where high-level decisions select options, skills, subgoals, or other abstractions that persist for multiple environment steps [14–16]. These methods improve long-horizon decision making by reducing the effective planning horizon and allowing the policy to reason over behavioral segments rather than individual low-level controls. More recent work on action chunking for RL shows that directly operating in a temporally extended action-sequence space can improve exploration and temporal-difference learning in long-horizon sparse-reward settings [17]. HiVA shares this temporal-abstraction perspective, but targets denoising-based VLA action experts rather than RL critics or separately learned option policies. In HiVA, a macro action is not a symbolic skill or independent high-level policy output; it is a duration-conditioned compact code predicted by the same action expert and decoded into native robot micro-actions.

Smooth and compact action representations. Another line of work studies action representations that are smoother or more compact than raw per-step action sequences. Diffusion policies and action-chunking transformers model multi-step action sequences directly, but they often operate in the raw action space and therefore inherit issues such as intra-chunk jitter, inter-chunk discontinuity, or sensitivity to the chosen horizon [1, 3, 11, 12]. Function-space representations address this issue by parameterizing trajectories with compact bases. BEAST encodes action sequences into B-spline tokens, showing that B-spline representations can compress action chunks and generate smooth high-frequency control signals efficiently [4]. ABPolicy predicts continuous B-spline control points with a flow-matching policy and uses asynchronous refitting to improve real-time smoothness and inter-chunk continuity [3].

HiVA builds on this evidence but uses B-splines differently. We relabel the training dataset into macro-actions and train

the model to learn the duration within which a B-spline code can accurately approximate the underlying micro-action sequence. Thus, the duration is not merely an inference-time heuristic; it is a supervised prediction target coupled to the compact trajectory code. We also avoid forcing all action dimensions into a single homogeneous representation. Translation and rotation are treated as separate smooth cumulative trajectories with different B-spline coefficient heads, while the gripper is modeled as a sparse event process with a discrete event-based token. This separation matches the geometry of the action space: end-effector translation and rotation are naturally smooth, whereas gripper commands are usually discontinuous open–close events.

Denoising-based VLA action experts. Recent VLAs increasingly use dedicated action experts that condition diffusion or flow-matching policies on multimodal visual-language features [1, 6, 7]. SmolVLA is a compact 0.45B VLA with a flow-matching action expert designed for efficient robotics, while VPP is a larger 1.5B-scale policy that conditions a diffusion policy on predictive visual representations from video modeling [6, 7]. HiVA is designed for this modular setting. Rather than replacing the pretrained multimodal conditioner or the main denoising transformer block, it changes the action-side interface: fixed raw-action tokens and heads are replaced with duration, translation-coefficient, rotation-coefficient, and gripper heads. This makes HiVA a hierarchical variable-horizon action representation that can be tested across different denoising VLA backbones and model scales.

2 Method

2.1 Overview

We replace direct micro-action generation with a macro-to-micro action expert. In general, a micro-action can be any robot-dependent native action vector $a_t \in \mathcal{A} \subset \mathbb{R}^m$, and a macro action is a duration-conditioned compact code $M_j = (d_j, z_j)$ that is decoded into an approximate variable-length sequence of native micro-actions $\hat{a}_{j,1:d_j} = \mathcal{D}_{d_j}(z_j)$. In this paper, we instantiate this framework for 7-DoF end-effector control, where z_j is represented by B-spline trajectory coefficients for translation and rotation plus an event-based gripper token. Specifically, for macro token j , the expert predicts

$$M_j = (d_j, \Theta_j^{\text{tr}}, \Theta_j^{\text{rot}}, g_j), \quad (1)$$

where $d_j \in \{1, \dots, D_{\max}\}$ is a discrete duration, Θ_j^{tr} are B-spline coefficients for chunk-relative cumulative translation, Θ_j^{rot} are B-spline coefficients for chunk-relative cumulative rotation, and g_j is an event-based gripper token. Micro-actions are reconstructed from these macro parameters rather than generated independently. Because the macro action is a lower-dimensional representation of the original micro-action sequence, the B-spline reconstruction is not expected to be exactly identical to the expert trajectory. Instead, it provides a

compact approximation whose reconstruction error is explicitly modeled during training. Our dynamic-programming re-labeling therefore favors segments that can be represented accurately by the B-spline code while still allowing a controlled approximation loss.

Figure 1 illustrates the motivation behind the macro-to-micro design. With a fixed-horizon action expert, every call commits to the same number of micro-actions regardless of local task difficulty. In easy free-space motion, this wastes model calls because the robot could safely move farther before replanning. Near the target object, however, the same fixed horizon can be too coarse: the policy may continue executing an outdated chunk, overshoot the desired grasp region, and fail the task. HiVA addresses this by making the action horizon part of the predicted macro action. The action expert can therefore output a long macro-action for simple approach motion and shorter macro-actions for delicate manipulation, balancing inference efficiency with local reactivity.

Figure 2 shows how HiVA is inserted into a denoising-based VLA. Instead of asking the action expert to predict a fixed horizon of raw micro-actions, we replace the action-token interface with macro-action tokens: a duration token that determines the number of micro-steps, translation and rotation coefficient tokens that describe smooth cumulative motion, and a gripper token that captures sparse open–close events. The corresponding output heads produce a macro action, and the B-spline basis expands the duration-conditioned coefficients into the final micro-action sequence. This design lets a single macro action represent a meaningful behavioral segment, while still preserving fine-grained translation, rotation, and gripper control inside that segment.

To make the approach easy to integrate into existing denoising-based VLAs, we only change a small number of lightweight layers and heads outside the main denoising transformer block in the action expert. Specifically, as summarized in Fig. 2, we keep the pretrained multimodal conditioning path and the main denoising transformer intact, and replace only the early action-side projection layers and the output heads. We instantiate this design in SmolVLA and VPP [6, 7]. In both cases, the raw fixed-horizon action interface is replaced by duration, coefficient, and gripper heads while the main pretrained denoising transformer remains unchanged.

2.2 Macro-action parameterization

For translation and rotation, the action expert predicts cubic B-spline coefficients in a chunk-relative cumulative representation. Let $\Phi_{d_j} \in \mathbb{R}^{d_j \times K}$ denote the cubic B-spline basis matrix for duration d_j . Translation is reconstructed as

$$\hat{P}_j = \Phi_{d_j} \Theta_j^{\text{tr}}, \quad (2)$$

where $\hat{P}_j(\tau)$ is the cumulative translation from the start of the chunk to local micro-step τ . Rotation is reconstructed similarly,

$$\hat{R}_j = \Phi_{d_j} \Theta_j^{\text{rot}}, \quad (3)$$

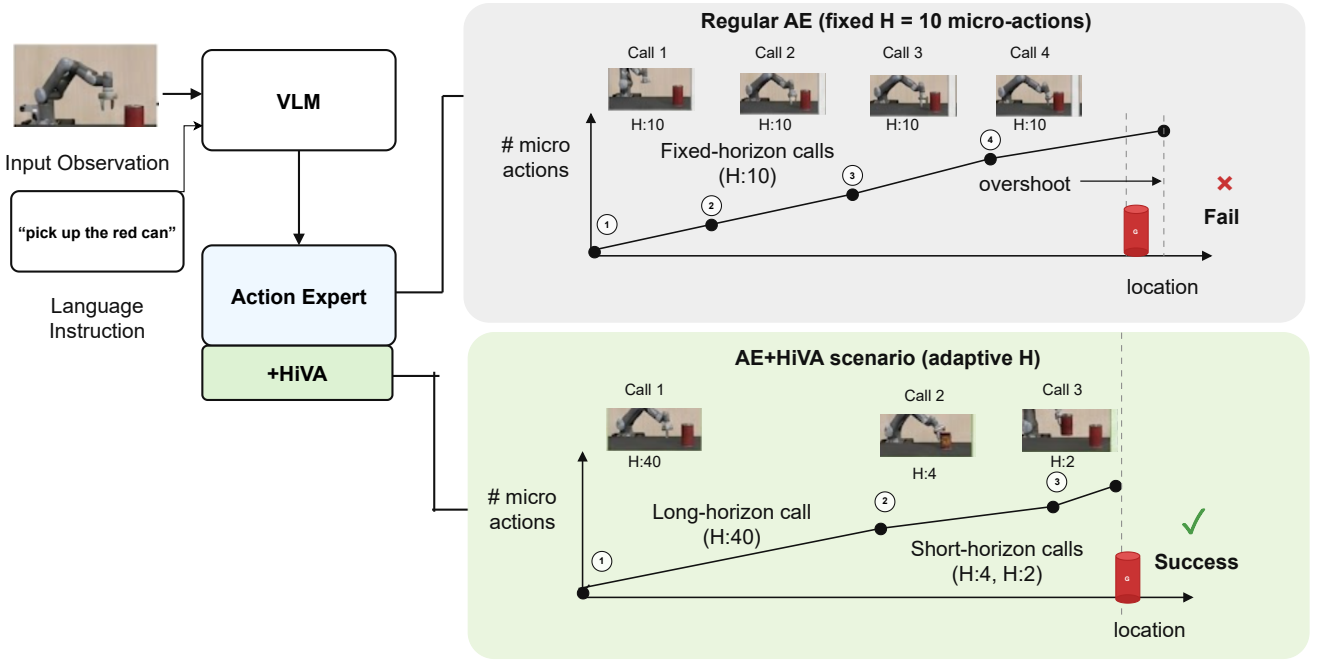


Figure 1: Overview of HiVA. A regular action expert (AE) executes a fixed number of micro-actions for every model call. This fixed-horizon design can waste calls during easy free-space motion and still remain insufficiently reactive near the target object, causing overshoot and task failure. AE+HiVA instead predicts variable-horizon macro actions: longer horizons for easy sub-trajectories and shorter horizons for difficult, contact-sensitive manipulation. This lets the policy spend fewer calls on simple motion while increasing reactivity near the object, leading to successful long-horizon execution.

where $\hat{R}_j(\tau)$ is the cumulative rotation vector relative to the chunk start.

The gripper is represented separately as a binary event process. We use one gripper token per macro action and predict an event type $e_j \in \{\text{hold}, \text{flip}\}$ together with an optional switch time $\tau_j \in \{1, \dots, D_{\max}\}$. This separation is intentional: translation and rotation are smooth geometric trajectories, while the gripper sequence is sparse and event-like.

2.3 Joint duration reasoning with a denoising action expert

Duration and coefficients are reasoned jointly by the same transformer block, but they use different output types. The duration token is supervised with a bounded discrete head over $\{1, \dots, D_{\max}\}$, while coefficient tokens are optimized with a continuous denoising objective, instantiated as flow matching or diffusion denoising depending on the host action expert. This design lets duration and coefficients interact through self-attention without forcing a discrete variable into the same continuous objective.

During training, the predicted duration distribution also participates in a soft duration-aware reconstruction. Instead of selecting a single hard predicted duration, we form a weighted mixture over hold-padded B-spline bases, so duration errors directly affect the reconstructed cumulative trajectories. This gives the model a clean way to learn that underestimating duration flattens a chunk too early, while overestimating duration

causes unnecessary motion after the ground-truth chunk ends.

2.4 Compact training objective

Let J be the number of macro tokens in a training sample. The total loss is

$$L_{\text{total}} = \frac{1}{J} \sum_{j=1}^J L_j, \quad (4)$$

with

$$\begin{aligned} L_j = & \lambda_{\text{dur}} L_{\text{dur},j} + \lambda_{\text{den}}^{\text{tr}} L_{\text{den},j}^{\text{tr}} + \lambda_{\text{path}}^{\text{tr}} L_{\text{path},j}^{\text{tr}} \\ & + \lambda_{\text{tail}}^{\text{tr}} L_{\text{tail},j}^{\text{tr}} + \lambda_{\text{den}}^{\text{rot}} L_{\text{den},j}^{\text{rot}} + \lambda_{\text{path}}^{\text{rot}} L_{\text{path},j}^{\text{rot}} \\ & + \lambda_{\text{tail}}^{\text{rot}} L_{\text{tail},j}^{\text{rot}} + \lambda_{\text{evt}} L_{\text{evt},j} + \lambda_{\text{time}} L_{\text{time},j} \\ & + \lambda_{\text{path}}^{\text{grip}} L_{\text{path},j}^{\text{grip}} + \lambda_{\text{tail}}^{\text{grip}} L_{\text{tail},j}^{\text{grip}}. \end{aligned} \quad (5)$$

Here L_{dur} supervises the duration categorical distribution, $L_{\text{den}}^{\text{tr}}$ and $L_{\text{den}}^{\text{rot}}$ are the native denoising objectives on translation and rotation coefficients, L_{path} and L_{tail} measure reconstruction error on the valid chunk prefix and the hold-padded suffix, and the gripper losses supervise event type, switch time, and the reconstructed binary gripper state. Detailed definitions are provided in the appendix.

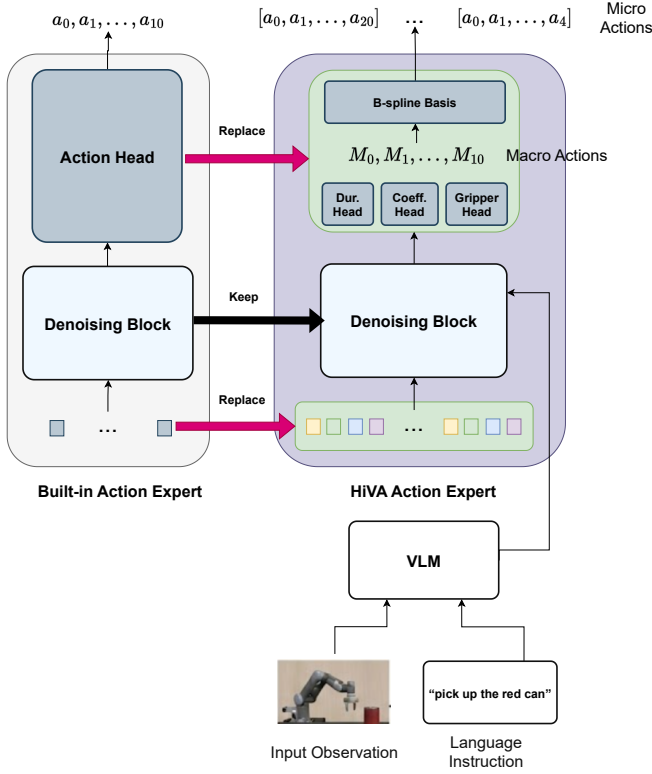


Figure 2: HiVA architecture. HiVA replaces the fixed raw-action interface of a denoising VLA action expert with a macro-action interface. The action tokens are decomposed into duration, translation-coefficient, rotation-coefficient, and gripper tokens, allowing each macro action to capture both a behavioral segment and fine-grained control within that segment. The built-in action head is replaced by duration, coefficient, and gripper heads, and a B-spline basis converts the predicted coefficients and duration into smooth variable-length micro-actions.

3 Training Dataset and Evaluation Benchmarks

3.1 Training dataset construction

We relabel each expert trajectory into macro-actions with a dynamic-programming (DP) oracle. Consider a candidate segment that starts at micro-step s and spans d micro-actions. Its cost is

$$C_{s,d} = \gamma + \alpha E_{s,d} + \beta R_{s,d}^{\text{evt}}, \quad (6)$$

where $E_{s,d}$ is a B-spline fit error and $R_{s,d}^{\text{evt}}$ is an event-proximity risk.

For a candidate segment, we construct chunk-relative cumulative translation and rotation targets from the expert trajectory. Let p_t and q_t denote the observed end-effector position and quaternion at micro-step t . Then the cumulative translation target for local index $\tau \in \{1, \dots, d\}$ is

$$P_{s,d}^*(\tau) = p_{s+\tau-1} - p_{s-1}, \quad (7)$$

and the cumulative rotation target is written abstractly as

$$R_{s,d}^*(\tau) = f_{\text{quat} \rightarrow \text{axis}}(q_{s+\tau-1}, q_{s-1}), \quad (8)$$

where $f_{\text{quat} \rightarrow \text{axis}}$ denotes the benchmark-provided conversion from the boundary quaternions to a chunk-relative axis-angle vector; the LIBERO-Plus instantiation is given in the appendix. A cubic B-spline is then fit to each candidate segment by regularized least squares, and the fit error is computed from the resulting reconstruction error.

To discourage macro-actions from crossing delicate event boundaries, we define a per-step event risk. Let e_i and e_{i+1} denote two consecutive gripper event times, and let $L_i = e_{i+1} - e_i$. For a micro-step u with $e_i \leq u < e_{i+1}$, we define a normalized pre-event term

$$r_{u,i}^{\text{pre}} = \frac{\alpha_{\text{pre}}}{\frac{(e_{i+1}-1)-u}{\max(1, L_i-1)} + \delta}, \quad (9)$$

and a delayed post-event bump

$$r_{u,i}^{\text{post}, \rho} = \alpha_{\text{post}} \mathbf{1}[u \geq e_i + \rho] \exp\left(-\frac{u - (e_i + \rho)}{\tau_{\text{post}}}\right), \quad (10)$$

with $\tau_{\text{post}} = 2$. The segment-level event risk is aggregated with log-sum-exp:

$$R_{s,d}^{\text{evt}} = \frac{1}{\kappa} \log \sum_{u=s}^{s+d-1} \exp(\kappa(r_{u,i(u)}^{\text{pre}} + r_{u,i(u)}^{\text{post}, \rho})), \quad (11)$$

where $i(u)$ is the most recent gripper event index such that $e_{i(u)} \leq u$.

We impose the gripper rule as a separate hard feasibility constraint rather than folding it into the cost. Let $\nu_{s,d}$ be the number of gripper flips in the candidate segment and let $\ell_{s,d}$ be the number of micro-steps after the first flip inside the same segment. Then the candidate is infeasible when

$$\nu_{s,d} > 1 \quad \text{or} \quad \ell_{s,d} > \rho, \quad (12)$$

and we set

$$C_{s,d} = +\infty. \quad (13)$$

This ensures that each macro-action contains at most one gripper event and at most ρ settle steps after that event.

The optimal segmentation is obtained by dynamic programming:

$$F[0] = 0, \quad F[t] = \min_{d \in \mathcal{D}, d \leq t} (F[t-d] + C_{t-d+1,d}). \quad (14)$$

Backtracking from $F[T]$ yields the final macro boundaries and therefore the GT duration labels d_j^* . For each selected segment, we retain the fitted translation and rotation coefficients, the cumulative translation and rotation targets, the binary gripper sequence, the gripper start state, and the event/timing labels used by the action expert.

3.2 Benchmarks

We use two long-horizon manipulation benchmarks. LIBERO-Plus extends the original LIBERO benchmark with controlled perturbations over object layout, camera viewpoint, robot initial state, language, lighting, background, and sensor noise, exposing substantially lower and more diagnostic performance for existing VLA models than the original LIBERO suites [8, 9]. We therefore target LIBERO-Plus rather than the original LIBERO benchmark to stress-test robustness under realistic distribution shifts. CALVIN is explicitly designed for long-horizon language-conditioned manipulation and is commonly evaluated by the average number of chained tasks completed successfully in sequence [10]. We use LIBERO-Plus and CALVIN because both require the policy to maintain consistency across multi-phase manipulation sequences rather than solve isolated one-step reactions.

3.3 Backbones and protocol

We evaluate the proposed action expert in two representative denoising-based VLAs with different model-size scales: SmolVLA (0.45B) and VPP (1.5B) [6, 7]. In both cases, we keep the main pretrained denoising transformer block and the multimodal conditioning path, and replace only the small action-side layers surrounding that block. SmolVLA is evaluated on LIBERO-Plus, while VPP is evaluated on CALVIN.

The main comparisons are against the original raw-action experts of the host architectures. For LIBERO-Plus, we compare SmolVLA against SmolVLA+HiVA(ours), using the existing SmolVLA task-success baseline of 69.0% as the reference. For CALVIN, we compare VPP against VPP+HiVA(ours), using the existing VPP average completed sequence length of 4.33 in the full-data setting and 3.25 in the data-efficiency setting as references. We will also report the number of model calls per episode and total completion time to measure whether variable-length macro-actions reduce inference cost while preserving or improving success.

A Detailed Training Objective

This appendix expands the compact loss in Eq. (5). Throughout the appendix, $D_{\max}$ is the maximum macro duration and K is the number of spline basis functions.

A.1 LIBERO-Plus cumulative rotation labels

In the LeRobot LIBERO-Plus environment, the chunk-relative cumulative rotation target can be instantiated with the benchmark-provided quaternion utilities. Let q_t denote the observed end-effector quaternion at micro-step t . For a candidate segment (s, d) and local index $\tau \in \{1, \dots, d\}$, we define the relative quaternion

$$q_{s,d}^{\text{rel}}(\tau) = \text{quat_distance}(q_{s+\tau-1}, q_{s-1}), \quad (15)$$

and convert it to a rotation vector by

$$R_{s,d}^*(\tau) = \text{quat2axisangle}(q_{s,d}^{\text{rel}}(\tau)). \quad (16)$$

This is the LIBERO-Plus instantiation of the abstract mapping $f_{\text{quat} \rightarrow \text{axis}}$ used in the main text.

A.2 Duration loss

Let the duration token produce logits

$$z_j^{\text{dur}} = W_{\text{dur}} h_j^{\text{dur}} \in \mathbb{R}^{D_{\max}}. \quad (17)$$

The duration distribution is

$$\pi_{j,d} = \text{softmax}(z_j^{\text{dur}})_d, \quad d \in \{1, \dots, D_{\max}\}. \quad (18)$$

The duration loss is cross-entropy plus an ordinal penalty:

$$L_{\text{dur},j} = -\log \pi_{j,d_j^*} + \lambda_{\text{ord}} \sum_{d=1}^{D_{\max}} \pi_{j,d} \left(\frac{d - d_j^*}{D_{\max} - 1} \right)^2. \quad (19)$$

A.3 Soft duration-aware reconstruction

For each duration d , let $\Phi_d \in \mathbb{R}^{d \times K}$ be the cubic B-spline basis matrix. To make all candidate durations compatible inside one fixed-size tensor, we define a hold-padded basis matrix $\tilde{\Phi}_d^{\text{hold}} \in \mathbb{R}^{D_{\max} \times K}$ by

$$\tilde{\Phi}_d^{\text{hold}}(\tau, :) = \begin{cases} \Phi_d(\tau, :), & \tau \leq d, \\ \Phi_d(d, :), & \tau > d. \end{cases} \quad (20)$$

The soft duration-aware basis mixture is then

$$\bar{\Phi}_j = \sum_{d=1}^{D_{\max}} \pi_{j,d} \tilde{\Phi}_d^{\text{hold}}. \quad (21)$$

Let $\hat{\Theta}_j^{\text{tr}}, \hat{\Theta}_j^{\text{rot}} \in \mathbb{R}^{K \times 3}$ denote the predicted translation and rotation coefficients. The reconstructed cumulative trajectories are

$$\hat{P}_j = \bar{\Phi}_j \hat{\Theta}_j^{\text{tr}}, \quad \hat{R}_j = \bar{\Phi}_j \hat{\Theta}_j^{\text{rot}}. \quad (22)$$

For a GT macro duration d_j^* , define hold-padded cumulative GT targets

$$\tilde{P}_j^*(\tau) = \begin{cases} P_j^*(\tau), & \tau \leq d_j^*, \\ P_j^*(d_j^*), & \tau > d_j^*, \end{cases} \quad (23)$$

$$\tilde{R}_j^*(\tau) = \begin{cases} R_j^*(\tau), & \tau \leq d_j^*, \\ R_j^*(d_j^*), & \tau > d_j^*. \end{cases} \quad (24)$$

Let $m_{j,\tau} = \mathbf{1}[\tau \leq d_j^*]$ and $M_j = m_j \mathbf{1}_3^\top$. Then

$$L_{\text{path},j}^{\text{tr}} = \frac{1}{d_j^*} \left\| M_j \odot (\hat{P}_j - \tilde{P}_j^*) \right\|_1, \quad (25)$$

$$L_{\text{tail},j}^{\text{tr}} = \frac{1}{\max(1, D_{\max} - d_j^*)} \left\| (1 - M_j) \odot (\hat{P}_j - \tilde{P}_j^*) \right\|_1. \quad (26)$$

The rotation path and tail losses are defined analogously:

$$L_{\text{path},j}^{\text{rot}} = \frac{1}{d_j^*} \left\| M_j \odot (\hat{R}_j - \tilde{R}_j^*) \right\|_1, \quad (27)$$

$$L_{\text{tail},j}^{\text{rot}} = \frac{1}{\max(1, D_{\max} - d_j^*)} \left\| (1 - M_j) \odot (\hat{R}_j - \tilde{R}_j^*) \right\|_1. \quad (28)$$

A.4 Continuous denoising objective for translation and rotation coefficients

The coefficient branch uses the native continuous denoising objective of the host action expert. We therefore write the main translation and rotation losses in the generic form $L_{\text{den},j}^{\text{tr}}$ and $L_{\text{den},j}^{\text{rot}}$.

Flow-matching instantiation. For flow-matching backbones, let $\Theta_j^{\text{tr},*}$ be the GT translation coefficient matrix and sample $u \sim \mathcal{U}[0, 1]$, $\epsilon_j^{\text{tr}} \sim \mathcal{N}(0, I)$. Define

$$\Theta_j^{\text{tr},u} = u\Theta_j^{\text{tr},*} + (1-u)\epsilon_j^{\text{tr}}, \quad (29)$$

and use the vector-field objective

$$L_{\text{den},j}^{\text{tr}} = \mathbb{E}_{u,\epsilon_j^{\text{tr}}} \left[\left\| v_{\theta}^{\text{tr}}(\Theta_j^{\text{tr},u}, u, c_j) - (\epsilon_j^{\text{tr}} - \Theta_j^{\text{tr},*}) \right\|_F^2 \right]. \quad (30)$$

The rotation version is identical with the superscript `rot`.

Diffusion instantiation. For diffusion backbones, one may instead sample a noise level t with cumulative schedule $\bar{\alpha}_t$ and define

$$\Theta_j^{\text{tr},t} = \sqrt{\bar{\alpha}_t} \Theta_j^{\text{tr},*} + \sqrt{1 - \bar{\alpha}_t} \epsilon_j^{\text{tr}}. \quad (31)$$

The translation denoising loss becomes

$$L_{\text{den},j}^{\text{tr}} = \mathbb{E}_{t,\epsilon_j^{\text{tr}}} \left[\left\| \epsilon_{\theta}^{\text{tr}}(\Theta_j^{\text{tr},t}, t, c_j) - \epsilon_j^{\text{tr}} \right\|_F^2 \right], \quad (32)$$

with the analogous rotation objective.

A.5 Gripper heads and reconstruction

We use a binary gripper event space $\{\text{hold}, \text{flip}\}$. Let the gripper token produce event logits

$$z_j^{\text{evt}} = W_{\text{evt}} h_j^{\text{grip}} \in \mathbb{R}^2, \quad (33)$$

with distribution

$$p_j^{\text{evt}} = \text{softmax}(z_j^{\text{evt}}). \quad (34)$$

The event loss is

$$L_{\text{evt},j} = -\log p_j^{\text{evt}}(e_j^*). \quad (35)$$

For switch time, let

$$z_j^{\text{time}} = W_{\text{time}} h_j^{\text{grip}} \in \mathbb{R}^{D_{\text{max}}}. \quad (36)$$

During training, mask logits beyond the GT duration:

$$\tilde{z}_{j,\tau}^{\text{time}} = \begin{cases} z_{j,\tau}^{\text{time}}, & \tau \leq d_j^*, \\ -\infty, & \tau > d_j^*. \end{cases} \quad (37)$$

Then

$$p_j^{\text{time}} = \text{softmax}(\tilde{z}_j^{\text{time}}), \quad C_{j,\tau} = \sum_{m=1}^{\tau} p_j^{\text{time}}(m). \quad (38)$$

The switch-time loss is defined only when the GT event is `flip`:

$$L_{\text{time},j} = \mathbf{1}[e_j^* = \text{flip}] (-\log p_j^{\text{time}}(\tau_j^*)). \quad (39)$$

Let $g_j^{\text{start}} \in \{0, 1\}$ be the gripper state at the start of macro j . During training we use the GT start state; during inference we use the observed current state for the first macro and the previous predicted final state for later macros. The predicted probability that the gripper is open at local micro-step τ is

$$\begin{aligned} \hat{g}_{j,\tau}^{\text{open}} &= p_j^{\text{evt}}(\text{hold}) g_j^{\text{start}} \\ &\quad + p_j^{\text{evt}}(\text{flip}) g_j^{\text{start}} (1 - C_{j,\tau}) \\ &\quad + p_j^{\text{evt}}(\text{flip}) (1 - g_j^{\text{start}}) C_{j,\tau}. \end{aligned} \quad (40)$$

Let the GT binary gripper sequence in macro j be $g_j^*(1), \dots, g_j^*(d_j^*)$. Hold-pad it by terminal extension:

$$\tilde{g}_j^*(\tau) = \begin{cases} g_j^*(\tau), & \tau \leq d_j^*, \\ g_j^*(d_j^*), & \tau > d_j^*. \end{cases} \quad (41)$$

Then the gripper path and tail losses are

$$L_{\text{path},j}^{\text{grip}} = \frac{1}{d_j^*} \sum_{\tau=1}^{d_j^*} \text{BCE}(\hat{g}_{j,\tau}^{\text{open}}, \tilde{g}_j^*(\tau)), \quad (42)$$

$$L_{\text{tail},j}^{\text{grip}} = \frac{1}{\max(1, D_{\text{max}} - d_j^*)} \sum_{\tau=d_j^*+1}^{D_{\text{max}}} \text{BCE}(\hat{g}_{j,\tau}^{\text{open}}, \tilde{g}_j^*(\tau)). \quad (43)$$

A.6 Complete macro loss

Combining all parts yields the loss quoted in the main paper:

$$\begin{aligned} L_j &= \lambda_{\text{dur}} L_{\text{dur},j} + \lambda_{\text{den}}^{\text{tr}} L_{\text{den},j}^{\text{tr}} \\ &\quad + \lambda_{\text{path}}^{\text{tr}} L_{\text{path},j}^{\text{tr}} + \lambda_{\text{tail}}^{\text{tr}} L_{\text{tail},j}^{\text{tr}} \\ &\quad + \lambda_{\text{den}}^{\text{rot}} L_{\text{den},j}^{\text{rot}} + \lambda_{\text{path}}^{\text{rot}} L_{\text{path},j}^{\text{rot}} \\ &\quad + \lambda_{\text{tail}}^{\text{rot}} L_{\text{tail},j}^{\text{rot}} + \lambda_{\text{evt}} L_{\text{evt},j} + \lambda_{\text{time}} L_{\text{time},j} \\ &\quad + \lambda_{\text{path}}^{\text{grip}} L_{\text{path},j}^{\text{grip}} + \lambda_{\text{tail}}^{\text{grip}} L_{\text{tail},j}^{\text{grip}}. \end{aligned} \quad (44)$$

B Preliminary HiVA-Lite Experiments

This appendix records the simplified first experiments used before the full DP-based HiVA system. The goal is to produce an interpretable preliminary result on LIBERO with minimal changes to SmolVLA: the model learns when to execute short, medium, or long prefixes, while the original raw action chunk training path remains largely unchanged.

B.1 Experiment 1: simple stable duration-query baseline

The first duration-only baseline keeps SmolVLA’s raw action representation and adds a learned neutral duration query to the

action expert. The query is not a ground-truth duration-class token. That is, the same learned token q_{dur} is appended for every training example, and the duration label d_s^* is used only as a cross-entropy target. This avoids label leakage while allowing the duration query to read the action-token representation produced by the action expert.

Let the duration set be

$$\mathcal{D} = \{1, 3, 8\}. \quad (45)$$

Given observation o_s , language instruction ℓ , and the host SmolVLA action expert, the model predicts an N -step raw action chunk

$$\hat{A}_s = (\hat{a}_s, \hat{a}_{s+1}, \dots, \hat{a}_{s+N-1}) \in \mathbb{R}^{N \times 7} \quad (46)$$

from the regular action-token outputs. The duration logits are predicted from the hidden state of the appended duration query,

$$z_s^{\text{dur}} \in \mathbb{R}^3, \quad \pi_s = \text{softmax}(z_s^{\text{dur}}), \quad (47)$$

where the three classes correspond to durations $[1, 3, 8]$.

During training, the original N -action flow-matching loss is retained. The duration head is supervised with a clean teacher-forced suffix pass. Specifically, the clean expert action chunk A_s^* is embedded with clean denoising time $t = 0$, the neutral query q_{dur} is appended after the N action tokens, and the duration head reads the resulting query hidden state:

$$z_s^{\text{dur-clean}} = W_{\text{dur}} h_{\text{dur}}(A_s^*, q_{\text{dur}}, t = 0, o_s, \ell). \quad (48)$$

The clean duration loss is

$$L_{\text{dur-clean}} = \text{CE}(z_s^{\text{dur-clean}}, d_s^*) = -\log \text{softmax}(z_s^{\text{dur-clean}})_{d_s^*} \quad (49)$$

The simple baseline objective is therefore

$$L_{\text{total}} = L_{\text{FM-action}} + 0.1 L_{\text{dur-clean}}. \quad (50)$$

Equivalently, this baseline uses $\lambda_{\text{dur}} = 0.1$.

At inference, SmolVLA first denoises the full raw action chunk as usual, producing $\hat{A}_s$. Then a final clean $t = 0$ suffix pass embeds the denoised chunk together with the same neutral duration query:

$$z_s^{\text{dur}} = W_{\text{dur}} h_{\text{dur}}(\hat{A}_s, q_{\text{dur}}, t = 0, o_s, \ell). \quad (51)$$

The predicted execution horizon is

$$\hat{d}_s = \arg \max_{d \in \mathcal{D}} \text{softmax}(z_s^{\text{dur}})_d, \quad (52)$$

and the controller executes only the first $\hat{d}_s$ actions before re-observing:

$$\hat{a}_s, \hat{a}_{s+1}, \dots, \hat{a}_{s+\hat{d}_s-1}. \quad (53)$$

Thus N remains the raw training and prediction chunk length, while $\hat{d}_s$ is the executed prefix length. This baseline is intentionally conservative: duration is learned from clean teacher-forced expert chunks during training and from the final denoised chunk during inference, without changing the normal action denoising trajectory.

Clean-plus-noisy duration-query ablation

The second duration-only baseline keeps the same neutral duration query but also trains it during noisy action-denoising passes. This variant is motivated by the architecture of flow-matching VLAs such as π_0 and SmolVLA, where noisy action tokens are processed together with clean visual, language, and state context inside an action expert [5, 6]. However, the exact duration-query auxiliary objective below is an ablation proposed for HiVA-lite rather than a loss copied directly from those codebases.

Let $\epsilon \sim \mathcal{N}(0, I)$ and let $t \in [0, 1]$ denote the denoising time under the convention used in this appendix, where $t = 0$ is clean and $t = 1$ is maximally noisy. A noisy action chunk is formed as

$$A_s^t = (1 - t)A_s^* + t\epsilon. \quad (54)$$

The duration query is appended to the noisy action suffix and passed through the same action-expert attention stack:

$$z_s^{\text{dur}}(t) = W_{\text{dur}} h_{\text{dur}}(A_s^t, q_{\text{dur}}, t, o_s, \ell). \quad (55)$$

The noisy duration loss is

$$L_{\text{dur-noisy}} = \mathbb{E}_{t, \epsilon} [w(t) \text{CE}(z_s^{\text{dur}}(t), d_s^*)], \quad (56)$$

where the time-dependent weight emphasizes near-clean inputs:

$$w(t) = \exp\left(-\frac{t^2}{\sigma^2}\right). \quad (57)$$

A practical starting range is $\sigma \in [0.25, 0.4]$. The weighting is important because duration labels should be predicted most confidently when the action representation is close to the final denoised action chunk; at very high noise, forcing a strong duration decision may encourage the query to ignore the action tokens and rely only on the observation prior.

The clean-plus-noisy objective is

$$L_{\text{total}} = L_{\text{FM-action}} + 0.05 L_{\text{dur-clean}} + 0.01 L_{\text{dur-noisy}}. \quad (58)$$

The clean loss anchors the duration head at the exact teacher-forced endpoint $t = 0$, while the noisy loss exposes the duration query to partially denoised action tokens resembling inference-time representations. If exact $t = 0$ is included in the sampling distribution and the same query, attention mask, and duration head are used, then $L_{\text{dur-noisy}}$ at $t = 0$ reduces to $L_{\text{dur-clean}}$. In practice, $L_{\text{dur-clean}}$ is kept as an explicit auxiliary term because continuous sampling of t rarely visits the exact clean endpoint.

Training proceeds as follows. First, the normal SmolVLA flow-matching branch samples t and ϵ , constructs A_s^t , and computes $L_{\text{FM-action}}$. Second, the model appends q_{dur} to the same noisy suffix and computes the weighted noisy duration cross-entropy in Eq. (56). Third, it performs an additional clean $t = 0$ teacher-forced duration pass using A_s^* to compute $L_{\text{dur-clean}}$. The duration query remains a neutral learned query in both passes; the ground-truth duration class is never embedded as an input token.

At inference, the query is appended throughout the denoising process. The action tokens are updated by the normal flow-matching solver, while the duration query is used only to produce duration logits. For a stable final prediction, we use the final denoised chunk and optionally average duration logits over the last few near-clean denoising evaluations:

$$\bar{z}_s^{\text{dur}} = \frac{1}{K} \sum_{k=1}^K z_s^{\text{dur}}(t_k), \quad t_k \approx 0. \quad (59)$$

The executed horizon is then

$$\hat{d}_s = \arg \max_{d \in \mathcal{D}} \text{softmax}(\bar{z}_s^{\text{dur}})_d, \quad (60)$$

and only the first $\hat{d}_s$ actions from the generated raw chunk are executed before the next observation. This advanced baseline gives the duration query access to the action denoising trajectory, while the small coefficient 0.01 on $L_{\text{dur-noisy}}$ limits the risk of disturbing the original action-generation objective.

B.2 Experiment 2: one-macro translation-coefficient HiVA-lite

The second preliminary experiment introduces a one-macro translation coefficient head while keeping rotation and gripper prediction in the regular SmolVLA raw action stream. Let $\mathcal{D} = \{1, 3, 8\}$, let $D_{\max} = \max \mathcal{D} = 8$, and let $K_{\max} = 4$. The model predicts three outputs: an N -step raw action chunk $\hat{A}_s^{\text{raw}}$, a duration distribution π_s , and translation coefficients $\hat{\Theta}_s^{\text{tr}} \in \mathbb{R}^{K_{\max} \times 3}$. For each $d \in \mathcal{D}$, let $\bar{\Phi}_d \in \mathbb{R}^{D_{\max} \times K_{\max}}$ denote the padded translation basis matrix: if $K(d) < K_{\max}$, unused columns are zero-padded, and for rows $\tau > d$ the final cumulative row is repeated so the decoded translation holds after duration d . The soft duration-aware mixed basis is

$$\bar{\Phi}_s^{\text{mix}} = \sum_{d \in \mathcal{D}} \pi_s(d) \bar{\Phi}_d. \quad (61)$$

The cumulative translation trajectory for the first $D_{\max}$ steps is then

$$\hat{P}_s^{\text{tr}} = \bar{\Phi}_s^{\text{mix}} \hat{\Theta}_s^{\text{tr}} \in \mathbb{R}^{D_{\max} \times 3}, \quad (62)$$

and the decoded translation micro-actions are obtained by finite differences,

$$\hat{u}_{s+\tau-1}^{\text{tr,coef}} = \hat{P}_s^{\text{tr}}(\tau) - \hat{P}_s^{\text{tr}}(\tau-1), \quad \tau = 1, \dots, D_{\max}, \quad \hat{P}_s^{\text{tr}}(0) = 0. \quad (63)$$

Let $\hat{A}_s^{\text{raw}} = (\hat{a}_s^{\text{raw}}, \dots, \hat{a}_{s+N-1}^{\text{raw}})$, where each raw action is decomposed as

$$\hat{a}_{s+\tau-1}^{\text{raw}} = [\hat{u}_{s+\tau-1}^{\text{tr,raw}}, \hat{u}_{s+\tau-1}^{\text{rot,raw}}, \hat{u}_{s+\tau-1}^{\text{grip,raw}}]. \quad (64)$$

During training, the assembled predicted action chunk $\bar{A}_s = (\bar{a}_s, \dots, \bar{a}_{s+N-1})$ uses coefficient-decoded translation on the first $D_{\max}$ tokens and the raw SmolVLA prediction afterwards:

$$\bar{a}_{s+\tau-1} = \begin{cases} [\hat{u}_{s+\tau-1}^{\text{tr,coef}}, \hat{u}_{s+\tau-1}^{\text{rot,raw}}, \hat{u}_{s+\tau-1}^{\text{grip,raw}}], & 1 \leq \tau \leq D_{\max}, \\ \hat{a}_{s+\tau-1}^{\text{raw}}, & D_{\max} < \tau \leq N. \end{cases} \quad (65)$$

For the GT duration d_s^* , define the GT cumulative translation prefix

$$P_s^*(\tau) = \sum_{m=1}^{\tau} u_{s+m-1}^{\text{tr,*}}, \quad \tau = 1, \dots, d_s^*, \quad (66)$$

then hold-pad it to $D_{\max}$ by

$$\tilde{P}_s^*(\tau) = \begin{cases} P_s^*(\tau), & \tau \leq d_s^*, \\ P_s^*(d_s^*), & d_s^* < \tau \leq D_{\max}, \end{cases} \quad \tilde{P}_s^*(0) = 0. \quad (67)$$

The hold-padded GT translation micro-actions for the first $D_{\max}$ steps are

$$\tilde{u}_{s+\tau-1}^{\text{tr,*}} = \tilde{P}_s^*(\tau) - \tilde{P}_s^*(\tau-1), \quad \tau = 1, \dots, D_{\max}. \quad (68)$$

The hybrid GT target chunk $\tilde{A}_s^* = (\tilde{a}_s^*, \dots, \tilde{a}_{s+N-1}^*)$ uses the padded GT translation for the first $D_{\max}$ tokens, the original GT translation from $D_{\max} + 1$ to N , and the GT rotation/gripper for all action tokens:

$$\tilde{a}_{s+\tau-1}^* = \begin{cases} [\tilde{u}_{s+\tau-1}^{\text{tr,*}}, u_{s+\tau-1}^{\text{rot,*}}, u_{s+\tau-1}^{\text{grip,*}}], & 1 \leq \tau \leq D_{\max}, \\ a_{s+\tau-1}^*, & D_{\max} < \tau \leq N. \end{cases} \quad (69)$$

The preliminary training loss is therefore

$$L_s^{\text{exp2}} = L_{\text{FM-action}}(\bar{A}_s, \tilde{A}_s^*) - \lambda_{\text{dur}} \log \pi_s(d_s^*). \quad (70)$$

This loss lets the FM action objective supervise a translation sequence whose first $D_{\max}$ tokens are reconstructed through the soft duration-aware coefficient branch, while the tail from $D_{\max} + 1$ to N remains the native SmolVLA raw prediction. During inference, a single macro is predicted, $\hat{d}_s = \arg \max_{d \in \mathcal{D}} \pi_s(d)$, the first $\hat{d}_s$ assembled actions are executed, and the policy then re-observes. For execution, the first $\hat{d}_s$ translation actions come from the coefficient branch, while rotation and gripper come from the regular raw action head.

B.3 Duration histograms and phase-conditioned statistics

Let $\mathcal{M}$ denote the set of labeled macro starts or dense training starts, and let each entry $j \in \mathcal{M}$ have duration label $d_j^* \in \mathcal{D}$. The macro-count duration histogram is

$$H_{\text{macro}}(d) = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \mathbf{1}[d_j^* = d]. \quad (71)$$

Since longer macros cover more control time, we also report a time-weighted histogram,

$$H_{\text{time}}(d) = \frac{\sum_{j \in \mathcal{M}} d_j^* \mathbf{1}[d_j^* = d]}{\sum_{j \in \mathcal{M}} d_j^*}. \quad (72)$$

For phase-conditioned analysis, each labeled segment (s_j, d_j^*) is assigned a pseudo-phase

$$\text{phase}(s_j, d_j^*) \in \{\text{free_motion}, \text{near_gripper}, \text{other}\}. \quad (73)$$

The conditional histogram is

$$H(d | \phi) = \frac{\sum_{j \in \mathcal{M}} \mathbf{1}[d_j^* = d, \text{phase}(s_j, d_j^*) = \phi]}{\sum_{j \in \mathcal{M}} \mathbf{1}[\text{phase}(s_j, d_j^*) = \phi]}. \quad (74)$$

These pseudo-phases are not provided by LIBERO; they are generated from gripper switch times. For $\mathcal{D} = \{1, 3, 8\}$, we use a near-event width $W_1 = 4$ and a transition width $W_3 = 6$. A segment is `near_gripper` if $[s_j, s_j + d_j^*)$ overlaps $[e - W_1, e + W_1)$ for any gripper event e . Given neighboring events $e_i \leq s_j < e_{i+1}$, it is `free_motion` if it lies fully inside the center region

$$[s_j, s_j + d_j^*) \subseteq [e_i + W_1 + W_3, e_{i+1} - W_1 - W_3). \quad (75)$$

All remaining segments are labeled `other`, corresponding mainly to approach, departure, and transition regions.

B.4 Basis size, effective degree, and control points

In the full coefficient-based HiVA representation, a basis matrix maps control coefficients to cumulative trajectories:

$$\hat{P} = \Phi_d \Theta^{\text{tr}}, \quad \Phi_d \in \mathbb{R}^{d \times K}, \quad \Theta^{\text{tr}} \in \mathbb{R}^{K \times 3}. \quad (76)$$

Here K is the number of basis functions, equivalently the number of control coefficients or control points per translation dimension. The coefficient matrix contains K 3D control vectors.

For very short durations, a fixed large cubic basis is underdetermined. We therefore define an effective active basis size

$$K_{\text{eff}}(d) = \min(d, K_{\text{max}}), \quad (77)$$

and an effective spline degree

$$p_{\text{eff}}(d) = \min(p_{\text{max}}, K_{\text{eff}}(d) - 1). \quad (78)$$

With $K_{\text{max}} = 4$ and $p_{\text{max}} = 3$, this gives

$$K_{\text{eff}}(d) = \min(d, 4), \quad p_{\text{eff}}(d) = \min(3, K_{\text{eff}}(d) - 1). \quad (79)$$

The first minimum prevents using more active control points than observed time steps; the second enforces the spline constraint $p \leq K - 1$.

For the preliminary $\mathcal{D} = \{1, 3, 8\}$ coefficient experiment, we use the simpler schedule

Duration d	$K(d)$	Degree $p(d)$	Interpretation
1	1	0	single-step constant action
3	2	1	linear transition macro
8	4	2	smooth free-motion macro

The duration-only experiment in Appendix B.1 does not use K directly; this schedule is reserved for the one-macro translation-coefficient variant in Appendix B.2.

B.5 Sidecar dataset format for duration-only HiVA-lite

The original LeRobot/LIBERO action trajectories remain unchanged. We store duration labels and diagnostics in a sidecar table, e.g., Parquet or JSONL for small-scale debugging. Each row corresponds to one dense training start s_j and contains pointers back to the original episode and frame. The current implementation also supports a small dataset-level metadata file, `meta/purecontact_pcf.json`, which stores verified pure-contact frame annotations. This metadata is read when building the sidecar; the expert action parquet files themselves are not rewritten.

The training sample returned by the dataset wrapper is

$$(o_{s_j}, \ell, A_{s_j:s_j+N-1}^*, d_j^*), \quad (80)$$

where $A_{s_j:s_j+N-1}^*$ is the standard N -step SmolVLA action target. The field `duration_class` is used by cross-entropy, while `duration_label` is used for histograms, logging, and inference-time conversion from class id to executed prefix length.

B.6 Horizon-safe dense duration labels

We train the duration-only model with an overlapping dense dataset: every eligible micro-action index s can become a training start. The label is assigned by a horizon-safe rule rather than by a non-overlapping macro segmentation. The current labeler applies the same rule to two kinds of interaction landmarks: explicit gripper open/close switch zones, derived from CMS and QPS; and pure-contact frame zones, derived from saved PCF annotations for contact without gripper open/close. The final label is the more conservative of the two labels.

Let $\mathcal{Z} = \{z_k\}_{k=1}^K$ be either the switch-zone set or the PCF-zone set. Each zone is an interval $z_k = [b_k, e_k]$. For explicit switches, $b_k = \min(c_k, q_k)$ and $e_k = \max(c_k, q_k)$, where c_k is the CMS frame and q_k is the QPS frame. For a PCF, the zone is a singleton, $b_k = e_k = p_k$. A frame inside any zone is always assigned duration 1. Otherwise, let $e_{\mathcal{Z}}^-(s)$ be the end of the previous zone, if one exists, and let $b_{\mathcal{Z}}^+(s)$ be the start of the next zone, if one exists. The beginning and ending of the trajectory are not treated as artificial switch zones. Thus, if no previous or next true interaction zone exists, the corresponding side is left open.

For a frame between two true neighboring zones, define the left and right near-target regions

$$\mathcal{N}_{\mathcal{Z}}^- = \{s : e_{\mathcal{Z}}^-(s) \leq s < e_{\mathcal{Z}}^-(s) + W_1\}, \quad \mathcal{N}_{\mathcal{Z}}^+ = \{s : b_{\mathcal{Z}}^+(s) - W_1 \leq s < b_{\mathcal{Z}}^+(s)\} \quad (81)$$

with $\mathcal{N}_{\mathcal{Z}} = \mathcal{N}_{\mathcal{Z}}^- \cup \mathcal{N}_{\mathcal{Z}}^+$. The center/free region is

$$\mathcal{C}_{\mathcal{Z}} = [e_{\mathcal{Z}}^- + W_1 + W_3, b_{\mathcal{Z}}^+ - W_1 - W_3). \quad (82)$$

In the implementation we use $\mathcal{D} = \{1, 3, 8\}$, $W_1 = 15$, and $W_3 = 15$. To keep the rule compact, define

$$R_3(s; \mathcal{Z}) = [s \notin \mathcal{N}_{\mathcal{Z}}] \wedge B_3(s; \mathcal{Z}), \quad (83)$$

$$R_8(s; \mathcal{Z}) = [s \in \mathcal{C}_{\mathcal{Z}}] \wedge B_8(s; \mathcal{Z}), \quad (84)$$

Sidecar field	Replay name	Meaning
dataset_index	–	global row index in the original LeRobot dataset
episode_index	EPS	source episode id
frame_index	FR	start micro-action index s_j
task_index	–	LIBERO dataset task id
libero_task_language	TK	language instruction displayed in the replay overlay
duration_label	D	final duration target $d_s^* = \min(d_s^S, d_s^P)$, with values in $\{1, 3, 8\}$
duration_class	–	CE class id, e.g., $1 \mapsto 0, 3 \mapsto 1, 8 \mapsto 2$
duration_label_s	Ds	duration from explicit gripper open/close switch zones only
duration_label_p	Dp	duration from pure-contact frame (PCF) zones only
phase	–	pseudo-phase: <code>near_target</code> , <code>approach</code> , or <code>free_motion</code>
dist_to_switch	DTS	distance to the nearest valid gripper switch zone endpoint
cmd_switch	CMS	command-side switch frame from the raw gripper action
qpos_switch	QPS	physical switch frame from gripper qpos hysteresis
pcf_frame	PCF	saved pure-contact frame for this episode, or -1 if absent
is_pcf	–	whether <code>frame_index</code> equals the saved PCF
dist_to_pure_contact	DTPC	distance to the nearest saved PCF
inside_purecontact_zone	–	whether the frame lies inside the PCF horizon-safe zone
purecontact_case_category	–	language-derived PCF case, e.g., <code>case1</code> or <code>case2</code>
purecontact_keyword	–	target object keyword used by the PCF language filter
purecontact_verb	–	manipulation verb used by the PCF language filter
labeler_version	–	relabeling rule/version identifier

Table 1: Sidecar fields and replay overlay names. The replay overlay currently displays TK, D, EPS, FR, Ds, DTS, CMS, QPS, Dp, PCF, and DTPC.

where

$$B_3(s; \mathcal{Z}) = [b_{\mathcal{Z}}^+ \text{ absent}] \vee [s + 3 \leq b_{\mathcal{Z}}^+ - W_1], \quad (85)$$

$$B_8(s; \mathcal{Z}) = [b_{\mathcal{Z}}^+ \text{ absent}] \vee [s + 8 \leq b_{\mathcal{Z}}^+ - W_1]. \quad (86)$$

A candidate duration d is horizon-safe at start s when

$$\text{valid}_{\mathcal{Z}}(s, d) = \begin{cases} 1, & d = 1, \\ \mathbf{1}[R_3(s; \mathcal{Z})], & d = 3, \\ \mathbf{1}[R_8(s; \mathcal{Z})], & d = 8. \end{cases} \quad (87)$$

Let $V_{\mathcal{Z}}(s) = \{d \in \mathcal{D} : \text{valid}_{\mathcal{Z}}(s, d) = 1\}$. The zone-specific label is

$$d_{\mathcal{Z}}(s) = \begin{cases} 1, & s \in \cup_k z_k, \\ \max V_{\mathcal{Z}}(s), & \text{otherwise.} \end{cases} \quad (88)$$

We compute d_s^S from the gripper switch zones and d_s^P from the saved PCF zones. If an episode has no PCF annotation, $d_s^P = 8$ for all s , so the PCF rule is inactive. The final dense duration label is

$$d_s^* = \min(d_s^S, d_s^P). \quad (89)$$

This produces overlapping samples such as

$$(o_s, \ell, A_{s:s+N-1}^*, d_s^*), \quad (o_{s+1}, \ell, A_{s+1:s+N}^*, d_{s+1}^*), \quad (90)$$

while preserving the distinction between the training chunk length N and the executed duration d_s^* . During inference, the model predicts N actions but executes only $\hat{d}_s \in \{1, 3, 8\}$ before re-observing.

B.7 Pure-contact frame detection and replay metadata

Not every contact-rich manipulation phase is marked by an explicit gripper open/close switch. In LIBERO drawer and microwave tasks, the gripper may hook, push, or drag the target

handle while the gripper command remains nearly constant. We therefore replace the earlier speed/curvature heuristic with a simulator-verified pure-contact frame (PCF) label. PCF is used only for relabeling and diagnostics; the expert action trajectory remains the original dataset trajectory.

We first select candidate tasks from `libero_task_language`. The current filters use pure-contact keywords `{drawer, microwave}` and verbs `{open, close, pull}`. We further divide candidates into two language cases. `case1` means the task first interacts with another object and then manipulates the target fixture, e.g., “put the mug in the microwave and close it.” `case2` means the target fixture is manipulated first, e.g., “open the drawer and put the bowl inside.” This distinction matters because `case1` often contains earlier incidental contacts before the intended fixture manipulation.

When PCF metadata must be refreshed, we replay the expert actions from the episode’s recorded LIBERO initial state and inspect MuJoCo contacts at each frame. A frame t is a PCF candidate if a gripper, finger, or palm geometry contacts the target drawer or microwave handle/door geometry. We then require that this contact be followed by sustained target joint motion. With window $H = 10$, movement threshold $\eta = 0.03$, and same-sign count $K = 10$, define

$$\Delta_t = \max_{1 \leq h \leq H} \|q_{t+h}^{\text{target}} - q_t^{\text{target}}\|_1, \quad (91)$$

and let n_t^{sign} be the larger of the positive and negative same-sign velocity counts in the next H target-joint velocity samples. The verified PCF onset is the first frame satisfying

$$\text{contact}_t = 1, \quad \Delta_t \geq \eta, \quad n_t^{\text{sign}} \geq K, \quad (92)$$

together with the task-phase gates implied by the language case. For `case1` close/open/pull tasks, the implementation additionally waits until the object-placement or release phase by requiring the gripper action not to be a close command at the candidate frame. The optional horizontal-motion gate is

disabled by default because drawer and microwave manipulation should not be filtered by vertical motion. The resulting metadata stores the onset pcf, the consecutive contact interval, the case category, keyword, verb, and detection method in `meta/purecontact_pcf.json`.

Normal replay generation and sidecar construction now use this saved PCF metadata. MuJoCo contact detection is rerun only when the metadata file itself needs to be updated. Thus, the fast replay path uses the saved pcf frame and original expert actions, while the slow update path recomputes contacts and rewrites `meta/purecontact_pcf.json`.

B.8 Gripper switch detection from LIBERO actions and qpos

We detect explicit gripper events using two complementary signals. The first is the raw LIBERO gripper command $a_t[-1]$, where $a_t[-1] < 0$ is an open command and $a_t[-1] > 0$ is a close command. This yields the command-side switch sequence CMS. The second is the gripper qpos pair stored in `state[-2:]`. We convert the two finger qpos values to a scalar width and apply hysteresis, producing the physical qpos-side switch sequence QPS.

The current implementation is more conservative than the initial pilot version. Same-type switches are merged with `merge_window=3`. CMS and QPS are then paired monotonically with the constraint $\text{CMS} \leq \text{QPS}$ and an absolute match window. Unpaired singletons are recovered rather than discarded: if CMS is present but QPS is absent, the QPS frame is chosen as the later frame whose measured gripper width is closest to the relevant hysteresis threshold; if QPS is present but CMS is absent, the CMS frame is synthesized three frames earlier. These recovered pairs define switch zones used by the horizon-safe duration rule.

```
import numpy as np

DURATIONS = (1, 3, 8)
W1 = 15
W3 = 15
MERGE_WINDOW = 3
MATCH_WINDOW = 20

def switch_times(binary_state):
    return np.where(binary_state[1:] != binary_state[:-1])
    [0] + 1

def command_state_from_libero_action(actions, delta=0.1):
    """
    actions: [T, 7] raw LIBERO actions.
    Returns close_state: 1=close command, 0=open command.
    Native assumption: action[-1] < 0 opens, action[-1] > 0
    closes.
    """
    u = actions[:, -1]
    close_state = np.zeros(len(u), dtype=np.int64)

    active = np.where(np.abs(u) > delta)[0]
    if len(active) == 0:
        return close_state

    first = active[0]
    close_state[:first] = int(u[first] > delta)
    for t in range(first, len(u)):
        if u[t] > delta:
            close_state[t] = 1
        elif u[t] < -delta:
```

```
            close_state[t] = 0
        else:
            close_state[t] = close_state[t - 1]
    return close_state

def gripper_width_from_state(state):
    """
    state: [T, 8]. The last two dimensions are gripper qpos.
    Use absolute values because the two finger joints may
    have opposite signs.
    """
    q_pair = state[:, -2:]
    return np.mean(np.abs(q_pair), axis=-1)

def open_state_from_qpos_hysteresis(state, low_pct=10,
                                     high_pct=90,
                                     alpha_closed=0.35,
                                     alpha_open=0.65):
    q_width = gripper_width_from_state(state)
    q_low = np.percentile(q_width, low_pct)
    q_high = np.percentile(q_width, high_pct)

    if q_high - q_low < 1e-6:
        open_state = np.zeros(len(q_width), dtype=np.int64)
        return open_state, q_width, (q_low, q_high, q_low,
                                      q_high)

    q_closed_thr = q_low + alpha_closed * (q_high - q_low)
    q_open_thr = q_low + alpha_open * (q_high - q_low)
    q_closed_thr = max(0.032, max(0.030, min(0.035,
                                              q_closed_thr)))
    q_open_thr = max(0.036, max(0.035, min(0.040,
                                             q_open_thr)))

    open_state = np.zeros(len(q_width), dtype=np.int64)
    midpoint = 0.5 * (q_low + q_high)
    open_state[0] = int(q_width[0] > midpoint)

    for t in range(1, len(q_width)):
        if q_width[t] > q_open_thr:
            open_state[t] = 1
        elif q_width[t] < q_closed_thr:
            open_state[t] = 0
        else:
            open_state[t] = open_state[t - 1]

    thresholds = (q_low, q_high, q_closed_thr, q_open_thr)
    return open_state, q_width, thresholds

def merge_same_type_switches(events, merge_window=
MERGE_WINDOW):
    events = np.array(sorted(set(map(int, events))), dtype=
np.int64)
    if len(events) == 0:
        return events
    merged = [int(events[0])]
    cluster_last = int(events[0])
    for e in events[1:]:
        e = int(e)
        if e - cluster_last <= merge_window:
            cluster_last = e
            continue
        merged.append(e)
        cluster_last = e
    return np.array(merged, dtype=np.int64)

def pair_switches_monotonic(cmd_switches, qpos_switches,
                             match_window=MATCH_WINDOW):
    # Dynamic-programming one-to-one matching. Maximize the
    # number of pairs,
    # then minimize total |CMS-QPS|. Only CMS <= QPS pairs
    # are valid.
    ...

def recover_singleton_switch_zones(zones, unmatched_cmd,
                                   unmatched_qpos,
                                   close_cmd, q_width,
                                   q_thresholds, T):
    # CMS without QPS: choose a later qpos frame closest to
```

```

    the target
    # hysteresis threshold. QPS without CMS: synthesize CMS
    at QPS-3.
    ...

def horizon_safe_duration_label(s, context, W1=W1, W3=W3):
    if context.inside_zone:
        return 1
    valid = [
        d for d in DURATIONS
        if not violates_phase_rule(
            s, d,
            prev_e=context.prev_switch_end,
            next_e=context.next_switch_start,
            has_prev_zone=context.has_prev_zone,
            has_next_zone=context.has_next_zone,
            W1=W1,
            W3=W3,
        )
    ]
    return max(valid) if valid else 1

def detect_switch_labels(actions, state):
    T = len(actions)

    close_cmd = command_state_from_libero_action(actions)
    cmd_switches = merge_same_type_switches(switch_times(
        close_cmd))

    open_qpos, q_width, q_thresholds =
        open_state_from_qpos_hysteresis(state)
    qpos_switches = merge_same_type_switches(switch_times(
        open_qpos))

    zones, unmatched_cmd, unmatched_qpos =
        pair_switches_monotonic(
            cmd_switches, qpos_switches
        )
    zones, unmatched_cmd, unmatched_qpos =
        recover_singleton_switch_zones(
            zones, unmatched_cmd, unmatched_qpos,
            close_cmd=close_cmd, q_width=q_width,
            q_thresholds=q_thresholds, T=T,
        )

    contexts = [frame_context_for_time(s, zones, T) for s
        in range(T)]
    duration_s = np.array([
        horizon_safe_duration_label(s, contexts[s], W1=W1,
        W3=W3)
        for s in range(T)
    ], dtype=np.int64)

    return {
        "switch_zones": zones,
        "cmd_switches": cmd_switches,
        "qpos_switches": qpos_switches,
        "q_width": q_width,
        "q_thresholds": q_thresholds,
        "duration_s": duration_s,
    }

```

References

- [1] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models. *arXiv preprint arXiv:2604.04161*, 2026.
- [2] Haoxuan Wang, Gengyu Zhang, Yan Yan, Ramana Rao Kompella, and Gaowen Liu. VLA knows its limits. *arXiv preprint arXiv:2602.21445*, 2026.
- [3] Fan Yang, Peiguang Jing, Kaihua Qu, Ningyuan Zhao, and Yuting Su. ABPolicy: Asynchronous B-spline flow policy for real-time and smooth robotic manipulation. *arXiv preprint arXiv:2602.23901*, 2026.
- [4] Hongyi Zhou, Weiran Liao, Xi Huang, Yucheng Tang, Fabian Otto, Xiaogang Jia, Xinkai Jiang, Simon Hilber, Ge Li, Qian Wang, Ömer Erdiñç Yağmurlu, Nils Blank, Moritz Reuss, and Rudolf Lioutikov. BEAST: Efficient tokenization of B-splines encoded action sequences for imitation learning. *NeurIPS*, 2025.
- [5] Kevin Black et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [6] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [7] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- [8] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- [9] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [10] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 2022.
- [11] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *Robotics: Science and Systems*, 2023.
- [12] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *Robotics: Science and Systems*, 2023.
- [13] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *NeurIPS*, 2025.

- [14] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- [15] Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. *NeurIPS*, 2018.
- [16] Andrew Levy, George Konidaris, Robert Platt, and Kate Saenko. Learning multi-level hierarchies with hindsight. *International Conference on Learning Representations*, 2019.
- [17] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. *arXiv preprint arXiv:2507.07969*, 2025.